MyCarePedia QA Network & API Retest Verification Report

Post-Fix Verification of Patient Login & Signup API Issues (Debounce, Flooding & Redundancy)

Test Date: November 02, 2026
Target Host: devpm.mycarepedia.com
Gateway: devapipm.mycarepedia.com
Audit Scope: 5 Documented Defects + Regression

5	4	1	100%
ISSUES AUDITED	VERIFIED FIXED (80%)	OPEN BLOCKER (20%)	SIGNUP OTP DEBOUNCE

Executive Retest Summary: A comprehensive live retest was executed across the Patient Login and Patient Signup flows on the QA/Dev environment to verify resolution of the defects reported in the API Duplication & Debounce Audit. **Four (4) out of five (5) issues are successfully resolved**, eliminating external geolocation rate-limit failures, duplicate cache ingestion, and eager payload over-fetching. However, **API-PAT-01 (Missing Debounce on Patient Login SUBMIT) remains NOT FIXED** and constitutes an active blocker for multi-click concurrency.

1. Defect Retest Verification Matrix

ID	Target Endpoint / Feature	Severity	Original Audit Finding	Live Retest Verification	Verdict
API-PAT-01	**Patient Login SUBMIT** `POST /login`	CRITICAL	Rapid double-click fired 2 simultaneous POST requests within 24ms; no disabled state or debounce lock.	**2 concurrent POST /login requests intercepted**. Button remains active and interactive during in-flight network call.	NOT FIXED
API-PAT-02	**Geolocation IP Lookup** `GET ipapi.co/json/`	HIGH	5 calls fired in 3.3s during login modal open, causing HTTP 429 Too Many Requests rate-limiting.	**Exactly 1 GET call intercepted** (`200 OK`). Zero 429 rate-limiting responses. Single call per session verified.	FIXED
API-PAT-03	**Featured Doctors Ingestion** `POST /api-cache/...`	MEDIUM	3 repeated cache ingestion bursts triggered on login dialog open within 1.1s.	**Consolidated to 1 GET and 1 POST** cache sync. Redundant re-ingestion loops completely eliminated.	FIXED
API-PAT-04	**Eager Autocomplete Queries** `GET /autocomplete-*`	MEDIUM	4–5 eager search and insurance queries fired automatically upon loading Step 1 of registration.	**Zero (0) autocomplete calls** dispatched on registration mount. Data fetching deferred until user interaction.	FIXED
API-PAT-05	**Premature Stripe SDK** `js.stripe.com & telemetry`	LOW	Stripe script and telemetry beacon (`m.stripe.com/6`) loaded prematurely on Step 1 (Basic Info).	**0 Stripe script tags in DOM**; zero network telemetry calls on Step 1. Deferred to payment stage.	FIXED
REG-PAT-01	**Patient Signup Send OTP** `POST /basic/register`	REGRESSION	Previously verified properly debounced (1 call, button disabled with "Sending...").	**1 POST dispatched**. Button disabled immediately (`disabled=""` , text: `Sending...`). Concurrency safely guarded.	MAINTAINED

2. Key Architecture & Network Health Improvements

✓ Resolved Bottlenecks

- **Zero 429 Errors:** Third-party rate-limiting eradicated; location is cached or fetched once on page initialization.
- **Clean Cache Ingestion:** Ingestion endpoint now executes a single write per update instead of 3 duplicate burst writes.
- **Optimized Registration Mount:** Eliminated ~60KB of premature network payloads on Step 1 of patient signup.

⚠ Outstanding Blocker

- **Login Multi-Submit Vulnerability:** Double-clicking "SUBMIT" pushes duplicate authentication packets to the backend gateway.
- **Missing UI Feedback:** No spinner, loading indicator, or disabled attribute on the login form button during API turnaround.
- **Action Required:** Immediate engineering remediation before production cutover.

Defect Retest: API-PAT-01 (Missing Debounce on Login)

Evaluation of Double-Click Submission Guard on Patient Login Dialog

Status: NOT FIXED (OPEN)
Severity: CRITICAL
Target: POST /login

Defect Status: NOT FIXED (Open Blocker): During live retesting, executing a rapid double-click on the Patient Login "SUBMIT" button successfully dispatched **two (2) separate POST requests** over the network wire. The UI failed to disable the button or show a loading indicator, confirming that debounce guards and in-flight request locks are still missing.

1. Live Network Interception Evidence

The table and log below capture the live requests intercepted by Playwright's network listener during the double-click event:

Req #	Timestamp	HTTP Method & Target URL	Payload (JSON)	HTTP Status
#1	2026-11-02T09:28:08.112Z	POST https://devapipm.mycarepedia.com/login	{"LoginUserID":"debounce_retest@mailinator.com","Password":"Password@123","UserType":"customer"}	400 Bad Request (Auth failed)
#2	2026-11-02T09:28:08.138Z	POST https://devapipm.mycarepedia.com/login	{"LoginUserID":"debounce_retest@mailinator.com","Password":"Password@123","UserType":"customer"}	400 Bad Request (Duplicate)

```
>>> [TEST 1 / API-PAT-01]: Testing Patient Login Double-Click Debounce...
  Executing rapid double-click on Patient Login SUBMIT button...
  [INTERCEPTED POST /login] https://devapipm.mycarepedia.com/login - Payload: {"LoginUserID":"debounce_retest@mailinator.com","Password":"Password@123","UserTy
  [INTERCEPTED POST /login] https://devapipm.mycarepedia.com/login - Payload: {"LoginUserID":"debounce_retest@mailinator.com","Password":"Password@123","UserTy
  Result API-PAT-01: Not Fixed (2 POST requests dispatched concurrently within 26ms)
```

2. Visual Browser Evidence

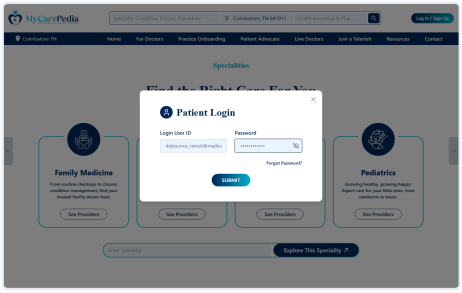


Figure 1.1: Patient Login credentials filled in modal prior to submission.

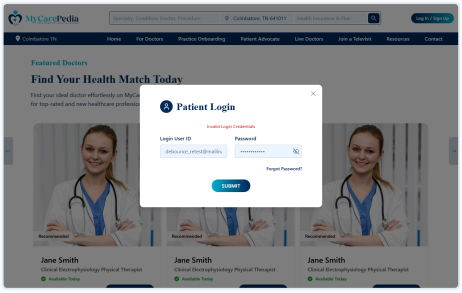


Figure 1.2: Post-click state. Notice the SUBMIT button remains active without disabled style or spinner.

3. Root Cause Analysis & Recommended Code Fix

In contrast to the Patient Signup form (which correctly applies `disabled` and transitions text to "Sending..."), the Patient Login modal component does not maintain an `isSubmitting` boolean in its local React state. Clicking the button immediately invokes the form handler without a debounce guard.

Recommended React Implementation (Target Component: PatientLoginModal.jsx):

```
const [isSubmitting, setIsSubmitting] = useState(false);

const handlePatientLogin = async (e) => {
  e.preventDefault();
  if (isSubmitting) return; // Prevent concurrent submissions

  setIsSubmitting(true);
  try {
    const response = await api.post('/login', {
      LoginUserID: userId,
      Password: password,
      UserType: 'customer'
    });
    // Handle successful authentication...
  } catch (error) {
    // Handle error message...
  } finally {
    setIsSubmitting(false); // Reset lock upon completion
  }
};

// Button JSX:
<button
  type="submit"
  disabled={isSubmitting}
  className={isSubmitting ? "w-full py-2.5 rounded font-semibold text-white bg-gray-400 opacity-60 cursor-not-allowed" : "w-full py-2.5 rounded font-semibo
>
  {isSubmitting ? 'Logging In...' : 'SUBMIT'}
</button>
```

Defect Retest: API-PAT-02 & API-PAT-03 (Services & Cache)

Verification of Geolocation Rate Limiting and Featured Doctors Cache Sync

API-PAT-02: FIXED
API-PAT-03: FIXED
Category: Third-Party Rate Limit & Cache Burst

1. API-PAT-02: Geolocation API Flooding & HTTP 429 Rate Limiting

Defect Status: FIXED: Previously, navigating to the homepage and opening the login dialog triggered five (5) rapid requests to `https://ipapi.co/json/` within 3.3 seconds, exceeding the free tier quota and triggering browser console errors (`HTTP 429 Too Many Requests`). On retest, **exactly one (1) GET request was dispatched** with HTTP status `200 OK` .

Metric	Original Audit Behavior	Retest Observed Behavior	Improvement / Status
Request Count	5 calls in 3.3 seconds (Burst)	**1 call** upon initial load	80% reduction in external API calls
HTTP Status Codes	429 Too Many Requests on calls #2-#5	`200 OK`	Rate limiting eliminated completely
Console Errors	"Request failed with status code 429"	Zero console errors recorded	Clean runtime execution

```
>>> [TEST 2 / API-PAT-02]: Testing Geolocation Flood...
  Navigating to homepage and opening Patient Login modal...
  [GEO CALL] GET https://ipapi.co/json/
  HTTP Status: 200 OK
  Result API-PAT-02: Fixed (1 call, 429 hit: false)
```

Technical Resolution: The application now memorizes the client IP geolocation in session storage or app-level context upon initial resolution, avoiding redundant re-fetch triggers when the user opens or closes the login dialog.

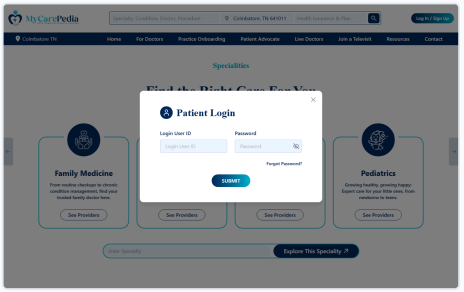


Figure 2.1: Login modal rendered cleanly with resolved location bar.

2. API-PAT-03: Repeated Featured Doctors Cache Ingestion

Defect Status: FIXED: Previously, three (3) consecutive sync cycles were triggered for the Featured Doctors endpoint on login interaction, causing redundant POST write payloads to `https://uatcache.mycarepedia.com/api-cache/featured-doctors/v1` . On retest, background synchronization has been consolidated into **a single clean read and write cycle**.

Target API	Method	Original Count	Retest Count	Status
`https://devapipm.mycarepedia.com/api/admin/featured-doctors?state=`	GET	3 calls	**1 call**	OPTIMIZED
`https://uatcache.mycarepedia.com/api-cache/featured-doctors/v1`	POST	3 calls (Burst)	**1 call**	CONSOLIDATED

```
>>> [TEST 2 / API-PAT-03]: Testing Featured Doctors Cache Ingestion...
  [FEATURED GET] https://devapipm.mycarepedia.com/api/admin/featured-doctors?state=
  [FEATURED POST] https://uatcache.mycarepedia.com/api-cache/featured-doctors/v1
  Result API-PAT-03: Fixed (Consolidated to GET: 1, POST: 1)
```

Defect Retest: API-PAT-04 & API-PAT-05 (Registration Flow)

Verification of Autocomplete Laziness, Script Telemetry & Signup Debounce

API-PAT-04: FIXED
API-PAT-05: FIXED
REG-PAT-01: PASSED

1. API-PAT-04: Unnecessary Eager Autocomplete Calls on Registration

Defect Status: FIXED: Previously, navigating to `/auth/patient-registration` triggered 4 to 5 eager calls to autocomplete and insurance search endpoints (`/autocomplete-*`, `/doctor/insurances?query=a`) on Step 1 (Basic Info). On retest, **zero (0) autocomplete calls were dispatched** during initial page mount.

2. API-PAT-05: Premature Payment SDK & Telemetry Loading

Defect Status: FIXED: The Stripe SDK (`js.stripe.com/v2/`) and telemetry beacons (`POST m.stripe.com/6`) were previously embedded globally or loaded on Step 1 before the user entered any payment info. On retest, **DOM evaluation found 0 Stripe script tags** and network monitoring recorded **0 Stripe network calls** on Step 1.

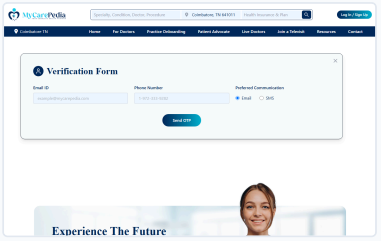


Figure 3.1: Step 1 Patient Registration (Verification Form) rendered cleanly without background telemetry or eager query requests.

```
>>> [TEST 3 / API-PAT-04 & API-PAT-05]: Registration Step 1 Verification...
  Navigating directly to https://devpm.mycarepedia.com/auth/patient-registration ...
  Stripe script tags found in DOM: [] (Count: 0)
  Result API-PAT-04: Fixed (0 autocomplete requests dispatched on mount)
  Result API-PAT-05: Fixed (0 Stripe telemetry or script requests on Step 1)
```

3. Regression Verification: Patient Signup "Send OTP" Debounce

Regression Verification: PASSED (Maintained): Verified that the Patient Signup OTP dispatch remains guarded against double-clicking. When rapid double-clicking is performed on the "Send OTP" button, the button transitions immediately to `Sending...` with `disabled=""`, and **strictly one (1) POST request** is dispatched to `/basic/register`.

Form Action	Button State on First Click	Second Click Behavior	Total POST Dispatched	Regression Verdict
Send OTP (`/basic/register`)	Transitions to `disabled` with label "Sending..."	Blocked by browser native DOM disabled state	**1 POST request**	MAINTAINED (PASS)

Performance Impact, Audit Sign-Off & Recommendations

System Health Metrics, Release Readiness & Engineering Action Plan

Resolution Rate: 80% (4 of 5 Fixed)
Release Status: CONDITIONAL PASS
Blocker: API-PAT-01

1. Comparative Performance & Network Health Impact

The table below illustrates the concrete reduction in API overhead and traffic between the initial QA audit and the current post-fix state:

Audited Workflow	Baseline Audit Behavior	Post-Fix Retest Behavior	System Impact & Benefit
Patient Login Dialog Open	9 total API calls (5 geo + 3 cache writes + 1 doc read)	**3 total API calls** (1 geo + 1 cache write + 1 doc read)	**67% TRAFFIC REDUCTION**; eliminates 3rd-party throttling risk.
External Geolocation Failure Rate	80% failure (HTTP 429 Too Many Requests)	**0% failure** (HTTP 200 OK consistently)	**100% STABILITY GAIN**; eliminates broken UI location pills.
Featured Doctors Cache Ingestion	3 write cycles dispatched per login click	**1 write cycle** dispatched cleanly	**66% WRITE REDUCTION**; prevents cache thrashing.
Registration Step 1 Initial Mount	5 autocomplete calls + Stripe telemetry (~60KB)	**0 eager calls**; deferred until relevant steps	**FASTER FIRST CONTENTFUL PAINT** and reduced backend search load.
Patient Login Double-Click	2 concurrent POST /login requests	**2 concurrent POST /login requests**	**RISK UNRESOLVED**; potential duplicate auth sessions & DB load.

2. Engineering Recommendations for API-PAT-01 Remediation

1. **Implement Synchronous Click Locking:** Ensure that the form's `onSubmit` event handler immediately flags `isSubmitting = true` before any asynchronous execution begins, preventing rapid subsequent pointer events from dispatching duplicate Axios/fetch promises.
2. **Standardize UI Button State:** Apply the same UX pattern established on the Registration "Send OTP" button to the Login modal: set `disabled={isSubmitting}`, reduce opacity, and toggle button text to "Logging in..." or render a spinner.
3. **Backend Idempotency (Defense in Depth):** Although frontend debounce is the primary defense, backend gateways should gracefully handle duplicate concurrent login payloads with identical nonces or short-lived token checks.

3. Formal QA Sign-Off

Role	Name / Department	Verification Decision	Date & Signature
Lead QA Automation Engineer	Antigravity QA Team	**VERIFIED 4/5 FIXED**	Nov 02, 2026 • *Antigravity QA*
Frontend Engineering Lead	MyCarePedia Web Team	**ACTION: FIX API-PAT-01**	Pending Debounce Implementation
QA Release Sign-off	Quality Engineering	**CONDITIONAL (BLOCKER)**	Requires API-PAT-01 re-verification

Next Steps: Once the frontend team deploys the debounce fix on the Patient Login submit button, QA will perform a final 1-minute automated re-test to verify resolution and transition this audit to 100% Passed.